

Grundlagen der C-Programmierung

Dr. Henrik Brosenne
Georg-August Universität Göttingen
Institut für Informatik

Wintersemester 2024/25

Strukturierung von Programmen

- Gültigkeitsbereiche von Namen

- Interne und externe Größen

- Das Modulkonzept

- Lokale und globale Größen

- Statische und automatische Variablen

Vereinbarungen

Wo in einem Programm dürfen Vereinbarungen stehen?

- In jedem Blocks dürfen Variablen deklariert werden.
- Funktionen müssen *global* (auf gleicher Ebene wie die Funktion `main`) definiert werden.
- Makros (mit `#define`) wurden jeweils vor die erste Funktion (das Hauptprogramm) geschrieben.
- Prototypen wurden ebenfalls vor die erste Funktion geschrieben.

Es gibt aber durchaus weitere Möglichkeiten.

- Variablen kann man auch außerhalb von Funktionen vereinbaren.
- Prototypen können innerhalb eines Blocks angegeben werden.

Beispiel

```
1  int i, j;
2
3  // invert string -----
4  void invert(char str[]) {
5      char h;
6      int length(char str[]);    // prototype
7
8      for (i = 0, j = length(str) - 1; i < j; i++, j--)
9          h = str[i], str[i] = str[j], str[j] = h;
10 }
11
12 // string length -----
13 int length(char str[]) {
14     for (i = 0; str[i] != '\0'; i++)
15         ;
16
17     return i;
18 }
```

Das Beispiel funktioniert nicht korrekt.

❶ Welchen Effekt hat ein Aufruf von `invert`?

```
1 char str[] = "Hello_World!";  
2 invert(str);  
3 printf("result:_'%s'\n", str);
```

❷ Warum passiert das?

❸ Wie könnte eine Korrektur aussehen?

Änderungen und Konsequenzen (1/2)

Der Prototyp der Funktion `length` wurde in die Funktion `invert` hineingezogen.

Namen, die innerhalb eines Blocks (also zum Beispiel im Rumpf einer Funktion) deklariert werden, sind nur innerhalb dieses Blocks bekannt.

Namen, die außerhalb von Funktionen deklariert werden, sind von der Stelle ihrer Deklaration bis zum Ende der Quelldatei bekannt.

Für das Beispiel hat das die Konsequenz, dass die Funktion `length` nur noch innerhalb der Funktion `invert` bekannt ist.

Ein andere Funktion, deren Definition weiter oben im Quellcode steht, könnte sie nicht aufrufen.

Änderungen und Konsequenzen (2/2)

Die Vereinbarungen der `int`-Variablen aus beiden Funktionen wurde herausgezogen und zu einer einzigen Vereinbarung zusammengezogen.

Konsequenz. Die `int`-Variablen `i` und `j` sind jetzt beiden Funktionen bekannt.

Was macht es aus, dass die Funktion `length` eine Variable (`j`) kennt, die sie nicht braucht?

Der Unterschied liegt im Bedeutungswandel des Namen `i`.

In der ursprünglichen Formulierung hatten beide Funktionen eine Variable `i`. Und diese Variablen hatten außer dem Namen nichts gemeinsam, insbesondere bezeichneten sie verschiedene Speicherplätze.

Jetzt teilen sich die beiden Funktionen eine Variable, die natürlich auch nur einen Speicherplatz besitzt mit der Folge, dass das Programm so nicht funktioniert.

Eine Größe heißt **intern**, wenn ihre Definition innerhalb einer Funktion steht, sonst **extern**.

Fazit

Bei Prototypen spricht viel dafür, sie gleich an den Anfang eines Programms zu schreiben, noch vor die erste Funktion, damit man sie bei Bedarf an beliebiger Stelle des Programms verwenden kann.

Wenn man Funktionen *verstecken* will, um ihre Verwendung auf ganz bestimmte Stellen zu beschränken, sollte man das Programm **modularisieren**.

Umgekehrt spricht bei Variablen viel dafür, sie möglichst lokal zu deklarieren, in der Regel also innerhalb einer Funktion.

Die Namen der Parameter in Prototypen und die Namen der Parameter in einer Funktionsdefinition sind nur innerhalb der Funktion bekannt.

Obwohl sowohl der Parameter von `invert` als auch der Parameter von `length` mit `str` bezeichnet wurde, haben die Parameter unterschiedliche Speicherstellen. Das im Beispiel in beiden Speicherstellen derselben Wert (Zeiger) gespeichert ist, ist Zufall.

Verschattung

Namen können **verschattet** werden, d.h. ein Name kann vorübergehend eine andere Bedeutung bekommen.

Genutzt haben wir das beim Prototyp von `length`. Der Name `str` bezeichnet innerhalb des Prototyps nicht den Parameter von `invert`, wie in den Zeilen davor und dahinter, sondern den Parameter von `length`.

In gleicher Weise verschattet jede Deklaration eines Namens innerhalb eines Blocks gleichnamige Größen außerhalb dieses Blocks.

```
1  int i;
2
3  for (i = 0; i < N; i++)
4  {
5      float i;
6      // ...
7  }
```

Strukturierung von Programmen

Gültigkeitsbereiche von Namen

Interne und externe Größen

Das Modulkonzept

Lokale und globale Größen

Statische und automatische Variablen

Erinnerung. Eine Größe heißt **intern**, wenn ihre Definition innerhalb einer Funktion steht, sonst **extern**.

Funktionen sind offenbar immer extern, da C die Schachtelung von Funktionsdefinitionen nicht erlaubt.

Bei Variablen hat man die Wahl.

- Interne Variablen stehen nur innerhalb des Blocks zur Verfügung, der ihre Definition enthält.
- Externe Variablen stehen allen Funktionen zur Verfügung, die hinter der Variablendefinition definiert werden – es sei denn, die Funktionen verschatten sie.

In der Regel sollte man versuchen, nur mit internen Variablen auszukommen. Es gibt aber durchaus auch Fälle, in denen externe Variablen zweckmäßig sind.

Beispiel Stapelverwaltung (1/6)

Für einen Stapelspeicher (Stack, *lifo* = *last in first out*) gibt es vier *Standardoperationen* in zwei Paaren.

- Bevor man ein Element auf den Stapel legen kann (*push*), muss man prüfen, ob noch Platz auf dem Stapel ist (*full*).
- Bevor man das oberste Element vom Stapel holen kann (*pop*), muss man prüfen, ob überhaupt ein Element auf dem Stapel liegt (*empty*).

Wie die Funktionen den Stapel verwalten, braucht und sollte den Benutzer der Funktionen nicht interessieren.

Beispiel Stapelverwaltung (2/6)

In diesem Beispiel wird der Stapel durch einen Vektor realisiert; denkbar wäre aber auch eine verkettete Liste.

Neben dem Vektor (`stack`) braucht man eine Indexvariable (`top`), die die erste freie Komponente des Vektors bezeichnet.

Da alle vier Funktionen auf Vektor und/oder Indexvariable operieren sollen, gibt es zwei Möglichkeiten.

- Vektor und/oder Indexvariable sind Parameter.

Das hat zur Folge, dass der Benutzer beide deklarieren und immer wieder als Argumente übergeben muss.

Die maximale Höhe des Stapels sollte bzw. müsste dann ebenfalls als Parameter übergeben werden.

- Vektor und Indexvariable werden als externe Variablen vereinbart.

Dann braucht man sie nicht mehr als Argumente übergeben.

Damit hält man sich auch noch die Möglichkeit offen die Funktionen jederzeit von der Vektorspeicherung auf eine verkettete Liste umstellen, ohne dass irgendwelche Änderungen an den Aufrufen vorzunehmen sind.

Beispiel Stapelverwaltung (3/6)

Für die konkrete Realisierung wird angenommen, dass Zeichen zu speichern sind. Ein Rahmenprogramm soll dem Benutzer die Möglichkeit geben, ein Zeichen auf den Stapel zu legen oder herunterzunehmen bzw. das Programm zu beenden.

```
1 // stack declarations *****
2 int full();
3 int empty();
4 void push(char c);
5 char pop();
```

Beispiel Stapelverwaltung (4/6)

```
1  // stack definitions -----
2  #define N 10
3
4  char stack[N];
5  int top = 0;
6
7  int full() {
8      return top >= N;
9  }
10
11 int empty() {
12     return top <= 0;
13 }
14
15 void push(char c) {
16     stack[top++] = c;
17 }
18
19 char pop() {
20     return stack[--top];
21 }
```

Beispiel Stapelverwaltung (5/6)

```
1 // auxiliary function *****
2 char readline();
3
4 // read first character of line -----
5 char readline() {
6     char c, t;
7     c = t = getchar();
8     while (t != '\n')
9         t = getchar();
10    return c;
11 }
```

Beispiel Stapelverwaltung (6/6)

```
1  int main() {
2      int loop = 1;
3      do {
4          printf ("0=push_character\n1=pop_character\n9=exit\ncmd>");
5          switch (readline()) {
6              case '0':
7              if (full())
8                  printf ("stack_is_full\n");
9              else {
10                 printf ("char>");
11                 push (readline());
12             }
13             break;
14             case '1':
15                 if (empty())
16                     printf ("stack_is_empty\n");
17                 else
18                     printf ("%c'was_top_character\n", pop());
19                 break;
20             case '9':
21                 loop = 0;
22                 break;
23             default:
24                 printf ("only_0,_1_or_9_are_valid_commands\n");
25             }
26         } while(loop);
27
28         // print stack ...
29         return 0;
30     }
```

Sind externe Variablen gelegentlich auch sehr nützlich, so sollte man sich doch von Fall zu Fall sehr genau überlegen, ob man sie verwendet oder nicht.

Bei extensiver Verwendung externer Variablen geht sehr schnell der Überblick verloren, wann wo was verändert wird – entsprechend sind nachträgliche Änderungen an den Funktionen extrem gefährlich.

Strukturierung von Programmen

Gültigkeitsbereiche von Namen

Interne und externe Größen

Das Modulkonzept

Lokale und globale Größen

Statische und automatische Variablen

Das Modulkonzept

Will man Abstrakte Datentypen, z.B. einen Stapelspeicher (Stapel), bereitstellen, die in unterschiedlichen Programmen für verschiedene Aufgaben genutzt werden können, müssen das nutzende Programm und die Funktionen (zur Stapelverwaltung) in verschiedenen Quelldateien stehen.

Wenn wir die Funktionen zur Stapelverwaltung in eine separate Quelldatei schreiben, etwa `stack.c`, können wir diese Datei wie folgt eingefügt werden.

```
#include "stack.c"
```

Die Anführungszeichen bewirken, dass der Präprozessor an anderen Stellen im Dateibaum sucht, als wenn spitze Klammern verwendet werden.

Dieses Vorgehen erweist sich bei der praktischen Arbeit an größeren Projekten schnell als unzureichend.

Man braucht eine Möglichkeit, Teile eines Programms voneinander unabhängig zu übersetzen und erst mit dem Linker zusammenzufügen.

Teile, die unabhängig voneinander übersetzt werden können, werden **Module** genannt.

Modul

C geht sogar noch einen Schritt weiter. Ein Modul, der nur Unterprogramme enthält, wird in zwei Quelldateien gespeichert.

- Eine Datei enthält nur die Deklarationen der Größen, die der Modul **exportiert**, also in der Regel Typdeklarationen und Prototypen von Funktionen, gelegentlich auch Konstanten und Variablen.

Solch eine Datei wird als **Headerdatei** bezeichnet.

- Die andere Datei enthält die Definitionen der zu exportierenden Größen, also insbesondere die Definitionen der Prototypen aus der Headerdatei.

Diese Datei kann aber auch zusätzliche lokale Funktionen realisieren, die dann nur innerhalb des Moduls verwendet werden können.

Auf die Headerdatei greift sie mit einer `include`-Direktive zu.

Üblicherweise erhalten Headerdatei und Implementation eines Moduls den selben Namen, bei der Headerdatei um `.h` und bei der Implementation um `.c` erweitert.

Will man auf einen separaten Modul zugreifen, muss man zwei Dinge tun.

- Dem Compiler muss man im **rufenden** Modul die Größen des **gerufenen** Moduls bekanntmachen.

Das geschieht mit einer `include`-Direktive für die Headerdatei des gerufenen Moduls.

- Dem Linker müssen die Module, die er zusammenbinden soll, einzeln genannt werden.

Beispiel, Headerdatei stack.h

```
1 // stack declarations *****
2 int full();
3 int empty();
4 void push(char c);
5 char pop();
```

Beispiel, Implementation stack.c

```
1  #include "stack.h"
2
3  // stack definitions -----
4  #define N 10
5
6  char stack[N];
7  int top = 0;
8
9  int full() {
10     return top >= N;
11 }
12
13 int empty() {
14     return top <= 0;
15 }
16
17 void push(char c) {
18     stack[top++] = c;
19 }
20
21 char pop() {
22     return stack[--top];
23 }
```

Beispiel, Testprogramm stack-demo.c (1/2)

```
1  #include <stdio.h>
2  #include "stack.h"
3
4  // auxiliary function *****
5  char readline();
6
7  //=====
8  int main() {
9      int loop = 1;
10     do {
11         printf ("0_=_push_ character\n"
12                "1_=_pop_ character\n"
13                "9_=_exit\n"
14                "cmd>_");
15         switch (readline()) {
16             // ...
17         }
18     } while(loop);
19
20     printf("stack\n");
21     while(!empty())
22         printf("%c\n", pop());
23     return 0;
24 }
```

Separate Compilation (1/2)

Wie übersetzt und bindet man Programme, die aus mehreren Modulen bestehen?

```
gcc -o stack-demo stack-demo.c stack.c
```

Konnten beide Module nacheinander fehlerfrei übersetzt werden, schließt sich der Lauf des Linkers unmittelbar an, so dass wir das ausführbare Programm erhalten.

Stellt der Compiler einen Fehler fest, etwa im Modul `stack-demo.c`, so übersetzt er zwar auch noch `stack.c` (oder versucht es zumindest); der Linker wird jedoch nicht mehr gestartet.

Der nächste Schritt ist nun natürlich, die Fehler zu beseitigen. Danach kann man dann neu übersetzen.

Für größere Projekte ist dieses Vorgehen ungeeignet. Bei jedem Compileraufruf zehn oder mehr Module hinschreiben ist umständlich, ganz davon abgesehen, dass es bei so vielen Modulen nicht besonders effizient ist, alle immer neu zu übersetzen.

Separate Compilation (2/2)

Günstiger ist es, die Objektdateien aufzubewahren und nur dann neu zu erzeugen, wenn sie nicht mehr auf dem aktuellen Stand sind.

Ein Modul muss neu

- wenn man an seiner Implementation etwas geändert hat.
- wenn an einer der Headerdateien, die er benutzt, eine Änderung vorgenommen wurde

Zum Beispiel könnte die Parameterliste einer Funktion verändert worden sein. Statt sich nun den Kopf darüber zu zerbrechen, wo die Funktion überall aufgerufen wird, lässt man den Compiler arbeiten. Er wird (wenn er dem Standard entspricht) jede Abweichung der Argumente eines Aufrufs von der (geänderten) Parameterliste bemerken.

Objektdateien erzeugen mit dem Compiler.

```
gcc -c stack.c
gcc -c stack-demo.c
```

Binden mit dem Linker.

```
gcc -o stack-demo stack-demo.o stack.o
```

Strukturierung von Programmen

Gültigkeitsbereiche von Namen

Interne und externe Größen

Das Modulkonzept

Lokale und globale Größen

Statische und automatische Variablen

Lokale und globale Größen

Das Beispiel zur modularisierten Lösung der Stapelverwaltung ist noch nicht ganz fertig. Es tut nämlich noch nicht wirklich das, was beschrieben wurde:

Die Variablen **stack** und **top** sind **noch nicht** im Modul „versteckt“!

Bei der Beschreibung der Gültigkeitsbereiche von Namen wurde das Modulkonzept noch völlig außer Acht gelassen.

Jede externe Größe eines Moduls ist, wenn man nicht ausdrücklich anderes bestimmt, gleichzeitig eine **globale** Größe, d.h. man kann auf sie auch aus anderen Modulen heraus zugreifen, egal ob sie in der zugehörigen Headerdatei deklariert wird oder nicht.

Interne Variablen sind dagegen gleichzeitig stets **lokale** Größen, d.h. man kann auf sie nur innerhalb der Funktion zugreifen, in der ihre Vereinbarung steht.

Ebenso sind benannte Konstanten, die in einer Implementation in einer `define`-Direktive definiert werden, stets **lokale** Größen.

Eine benannte Konstante, die in einer Headerdatei in einer `define`-Direktive definiert wird, steht allen Modulen zur Verfügung, die die Headerdatei verwenden.

Attribut static (1/2)

Es gibt eine einfache Möglichkeit, für externe Größen zu verhindern, dass sie gleichzeitig auch globale Größen sind.

Man fügt der Definition der Größe das Speicherklassen-Attribut **static** hinzu. Es ist üblich, wenn auch nicht notwendig, dieses Attribut an den Anfang der Definition setzen.

Entsprechend sollte die Datei `stack.c` wie folgt beginnen, damit die Variablen **stack** und **top** im Modul „versteckt“ werden.

```
1  #include "stack.h"
2
3  // stack definitions -----
4  #define N 10
5
6  static char stack[N];
7  static int top = 0;
8
9  // ...
```

Attribut static (2/2)

Ebenso wie Variablen kann man auch (Hilfs-)Funktionen mit dem Attribut **static** versehen, wenn diese Funktionen nur innerhalb des Moduls aufgerufen werden sollen.

Ein Beispiel dafür ist die Funktion **readline** im (Test-)Hauptmodul der Stapelverwaltung.

Wird **readline** mit dem Attribut **static** versehen, so ist sie im Modul „versteckt“; verzichtet man dagegen auf das Attribut **static**, so wird der Name **readline** an den Linker weitergegeben – und der meldet einen Fehler, falls in irgendeinem anderen der einzubindenden Module auch eine globale Variable oder Funktion mit dem Namen **readline** definiert ist.

Ebenso, wie man Variablen möglichst als interne Größen einer Funktion vereinbaren sollte, sollte man darauf achten, dass in jedem Modul alle Größen als lokal vereinbart werden, die nur innerhalb des Moduls (sinnvoll) verwendet werden können.

Man hat letztlich zwei voneinander unabhängige Ebenen des „Versteckens“, nämlich die Ebene des **Übersetzens** und die Ebene des **Bindens**.

Strukturierung von Programmen

Gültigkeitsbereiche von Namen

Interne und externe Größen

Das Modulkonzept

Lokale und globale Größen

Statische und automatische Variablen

Statische und automatische Variablen

Eine **statische** Variable steht prinzipiell während der gesamten Dauer der Ausführung eines Programms zur Verfügung, auch wenn man zeitweilig nicht auf sie zugreifen kann, weil sie in einigen Modulen nicht deklariert oder in einigen Funktionen verschattet ist.

Statisch sind alle externen Variablen unabhängig davon ob sie das Attribut **static** tragen oder nicht.

Für statische Variablen wird der Speicherplatz bereits durch den Compiler bereitgestellt und zugeordnet.

Der Compiler löscht die Speicherbereiche dieser Variablen auch mit binären Nullen, wenn der Programmierer nicht in der Definition explizit Anfangswerte, das muss ein **konstanter Ausdruck** sein, vorgibt.

Automatische Variablen

Das Pendant zu den statischen Variablen sind die **automatischen** Variablen.

Automatische Variablen sind interne Variablen, d.h. Variablen, die innerhalb eines Blocks (zum Beispiel im Rumpf einer Funktion) definiert werden. Diese Variablen besitzen nur einen Speicherplatz, während der Block ausgeführt wird, in dem sie definiert sind.

Einer automatischen Variablen wird in dem Moment ein Speicherplatz zugeordnet, in dem der Block betreten wird, der ihre Definition enthält; beim Verlassen des Blocks wird die Zuordnung wieder gelöst. Wird der Block erneut betreten, erfolgt eine neue Zuordnung allerdings unter Umständen zu einem anderen Speicherplatz.

Statische und automatische Variablen

Da der Standard schreibt nicht vor, dass automatische Variablen bei der Speicherplatzzuordnung einen wohldefinierten Anfangswert erhalten, deshalb **muss** man ihnen erst einen Wert zuweisen, bevor man auf ihren Wert zugreifen kann.

Diese Zuweisung kann bei automatische Variablen in der Form einer Anfangswertzuweisung erfolgen oder auch durch eine explizite Wertzuweisung, beides löst der Compiler in gleicher Weise auf.

```
1 void f(double a, double b) {  
2     double d = a * b;  
3     // ...  
4 }
```

oder

```
1 void f(double a, double b) {  
2     double d;  
3     d = a * b;  
4     // ...  
5 }
```

static

Variablen, die in einem Block definiert werden, dürfen das Attribut **static** erhalten.

Solche Variablen sind eine Mischung aus externen statischen Variablen und internen Variablen.

- Wie auf interne Variablen kann man auf sie nur in dem Block zugreifen, der ihre Definition enthält.

Das impliziert insbesondere, dass sie wie automatische Variablen lokal sind, also nicht aus anderen Modulen heraus gelesen oder verändert werden können.

- Wie bei externen Variablen erfolgt die Zuordnung des Speicherplatzes einmalig durch den Compiler, was entsprechend auch wieder nur **konstante Ausdrücke** als Anfangswerte bei der Definition erlaubt.

Beispiel (1/2)

Funktionen

```
1  int f() {
2      int z = 1;
3      return z++;
4  }
5
6  int g() {
7      static int z = 1;
8      return z++;
9  }
```

Aufruf

```
1  for (i = 1; i <= 3; i++)
2      printf("run %d: %d - %d\n", i, f(), g());
```

Ausgabe

```
run 1: 1 - 1
run 2: 1 - 2
run 3: 1 - 3
```

Beispiel (2/2)

Interpretation

- Bei **f** wird der Variablen **z** bei jedem Aufruf erneut ein Speicherplatz und diesem Speicherplatz der Anfangswert 1 zugeordnet, den **f** auch als Funktionswert liefert. Die Erhöhung des Wertes von **z** als Nebeneffekt bleibt letztlich ohne Wirkung, weil die Zuordnung der Variablen zu dem Speicherplatz mit dem Rücksprung wieder gelöst wird.
- Bei **g** wird der Variablen **z** bereits durch den Compiler ein Speicherplatz zugeordnet und mit dem Anfangswert 1 belegt. Bei jedem Aufruf liefert **g** als Funktionswert den Wert, den **z** gerade besitzt – und erhöht als Nebeneffekt den Wert von **z**. Da die Zuordnung zwischen der Variablen **z** und ihrem Speicherplatz mit dem Rücksprung **nicht** gelöst wird, hat **z** beim nächsten Aufruf den veränderten Wert des vorhergehenden Aufrufs.